Exp 7
03/04/26

Flow Control at Data Link Layer using sliding Window Protocol

Aim :-

To write a program to implement flow control at Data link layer using sliding window protocol and to simulate the flow of frames from one node to another.

Algorithm :-

1. Start the program
2. Read total no. of frames to be sent (N), sender window size (W) and frame no. to simulate as lost.
3. Initialize base = 0 & next seq = 0
4. Repeat while base < N :
   a. Determine current sender window as range [Base, Base + W - 1]
   b. Send all frames with seq. nos from next seq to (Base + W - 1), increment nextSeq after each frame.
   c. If lost frame lies within frames
      (i) Receiver discards lost frame
      (ii) Sender times out and retransmits all frames.
   d. else (no loss)
      (i) Receiver accepts each frame in order and sends cumulative acknowledgement for it

(iii) On receiving ACK for last frame sent, sender slides its window forward Base = next seq.

5. Repeat step 4 until all N frames transmitted.

6. Stop program.

Program :-

```
def simulate (total_frames, window_size,
              lost_frame):

    Base = 0
    next_seq = 0
    while Base < total_frames:
        window_end = min (Base + window_size
                          -1, total_frames -1)
        print (f" Sender Window : [Frame{base}
                - Frame { window_end }]")
        while next_seq < base + window_size and
            next_seq < total_frames:
            print (f" Node A --> Sending Frame
                   {next_seq}')
            if next_seq == lost_frame:
                print (f" (Frame {next_seq}^)
            next_seq += 1
        if lost_frame is not None and base <=
            lost_frame < next_seq:
            print (f" Node B --) Frame {lost_frame}
                   missing.")
            print (f" Node A --> Timeout ! Resending
                   from Frame { lost_frame }")
```

output :

Enter total no. of frames to send : 8

Enter sender's window size : 4

Enter frame no. to simulate as lost (0 to 7, enter -1 for no loss) : 5

Sender window : [Frame 0 - Frame 3]

Node A → Sending frame 0

Node A → Sending frame 1

Node A → Sending frame 2

Node A → Sending frame 3

Node B → received frame 0, Sending ACK 0

Node B → received frame 1, Sending ACK 1

Node B → received frame 2, Sending ACK 2

Node B → received frame 3, Sending ACK 3

Node A → ACK 3 received. Sliding window forward.

Sender window : [Frame 4 - Frame 7]

Node A → Sending frame 4

Node A → Sending frame 5

Frame 5 lost in transit

Node A → Sending frame 6

Node A → sending frame 7

Node B → Frame 5 missing. Discarding
out of order frames after
sending frame 5

Node A → Timeout resending window
starting from frame 5

Sender Window: [Frame 4 - Frame 7]

Node A → sending frame 5

Node A → sending frame 6

Node A → sending frame 7

Node B → received frame 4, sending
ACK 4

Node B → received frame 5, sending
ACK 5

Node B → received frame 6, sending
ACK 6

Node B → received frame 7, sending
ACK 7

Node A → ACK 7 received. Sliding
window forward.

All 8 frames successfully transmitted
from Node A to Node B.

```
        next_seq = lost_frame
        lost_frame = None
        continue
    for f in range(base, next_seq):
        print(f"Node B -> Received frame {f},
                                ACK {f}")
    print(f"Node A -> ACK {next_seq-1}
                received. sliding window")
    base = next_seq
print(f"All {total_frames}")

def main():
    print("sliding window protocol")
    total_frames = int(input("Enter total frames"))
    windowsize = int(input("Enter window size"))
    lost_frame = int(input(f"Enter lost
        frame (0 to {totalframes-1}, no loss):"))
    if lost_frame == -1:
        lost_frame = None
    simulate(total_frames, windowsize, lostfram

if __name__ == "__main__":
    main()
```

Result :-

Program to implement flow control s
data link layer using sliding wi
protocol was successfully executed.